



Universidad Tecnológica de Panamá
Centro Regional de Coclé
Facultad de Ingeniería en Sistemas Computacionales
Estructura de datos II

Proyecto Final

De: Álvaro Rodríguez, Fátima Calderón , Rafael Flores ,
Santiago Navarro , Roger Torrero, Boris Sánchez

Profesora: Ing. María Tejedor M.

Fecha: Martes, 7 de julio de 2026

Introducción

La movilidad urbana es un aspecto fundamental en las ciudades, ya que influye en los tiempos de traslado y el acceso a diferentes zonas. Para esta propuesta se utilizó un grafo para representar la red vial mediante nodos y conexiones, complementado con una estructura hash para gestionar la información asociada a cada punto del sistema.

Además, se incorporaron mecanismos de recorrido y cálculo de rutas para analizar las conexiones existentes y seleccionar caminos factibles dentro de la simulación. Con este proyecto se busca aplicar de manera práctica los conceptos de estructuras de datos estudiados durante el curso en un contexto relacionado con una situación real.

Descripción del proyecto

Este proyecto consiste en desarrollar un sistema para optimizar las rutas vehiculares en la zona central de Penonomé, sugiriendo recorridos con menor congestión. La red vial se representa mediante un grafo dirigido y ponderado, donde los nodos corresponden a puntos importantes de la ciudad y las aristas representan las rutas disponibles. Cada conexión posee un nivel de congestión generado aleatoriamente entre 0 y 1000 mediante la clase `Math.random()`.

Como estructura auxiliar se utiliza una tabla hash para almacenar y recuperar información de los nodos mediante identificadores únicos, empleando el método del residuo y sondeo cuadrático para resolver colisiones. Además, se utilizan las clases `Nodo` y `GrafoInterfaz` para representar los vértices y la visualización del sistema.

El objetivo es aplicar los conceptos de estructuras de datos en una simulación que permita analizar rutas y comprender el impacto del congestionamiento vehicular.

Justificación y objetivos

Esta investigación surge debido al congestionamiento vehicular en la zona central de Penonomé, especialmente en horas de alta circulación. Por ello, se propone un sistema que sugiera rutas menos congestionadas, apoyando la toma de decisiones para una mejor movilidad urbana y una circulación más eficiente.

Objetivo general

Desarrollar un sistema que permita a los usuarios encontrar rutas menos congestionadas en la zona central de Penonomé, reduciendo el tráfico y el tiempo de desplazamiento.

Objetivos específicos

- Diseñar un sistema que sugiera rutas alternativas con menor congestión vehicular.
- Implementar la visualización de rutas alternas.
- Codificar las estructuras necesarias del sistema, como grafo y tabla hash.

Marco teórico – estructura principal

¿Qué es un Grafo?

Un grafo es una estructura compuesta por **nodos** y **aristas** que permiten representar relaciones, caminos y conexiones entre diferentes elementos.

```
public class Grafo {
    private int maxNodos;
    private int numNodos;
    private Nodo[] nodos;
    private double[][] matrizAdyacencia;
    private boolean esDirigido;
    private EstructuraHash<Integer, Nodo> nodosHash;
    private EstructuraHash<Integer, Integer> indicePorId;

    // Constructor
    public Grafo(int maxNodos, boolean esDirigido) {
        this.maxNodos = maxNodos;
        this.numNodos = 0;
        this.esDirigido = esDirigido;
        this.nodos = new Nodo[maxNodos];
        this.matrizAdyacencia = new double[maxNodos][maxNodos];
        this.nodosHash = new EstructuraHash<>();
        this.indicePorId = new EstructuraHash<>();
        // inicializamos la matriz con infinito (sin conexión)
        for (int i = 0; i < maxNodos; i++) {
            for (int j = 0; j < maxNodos; j++) {
                matrizAdyacencia[i][j] = Double.POSITIVE_INFINITY;
            }
        }
    }
}
```

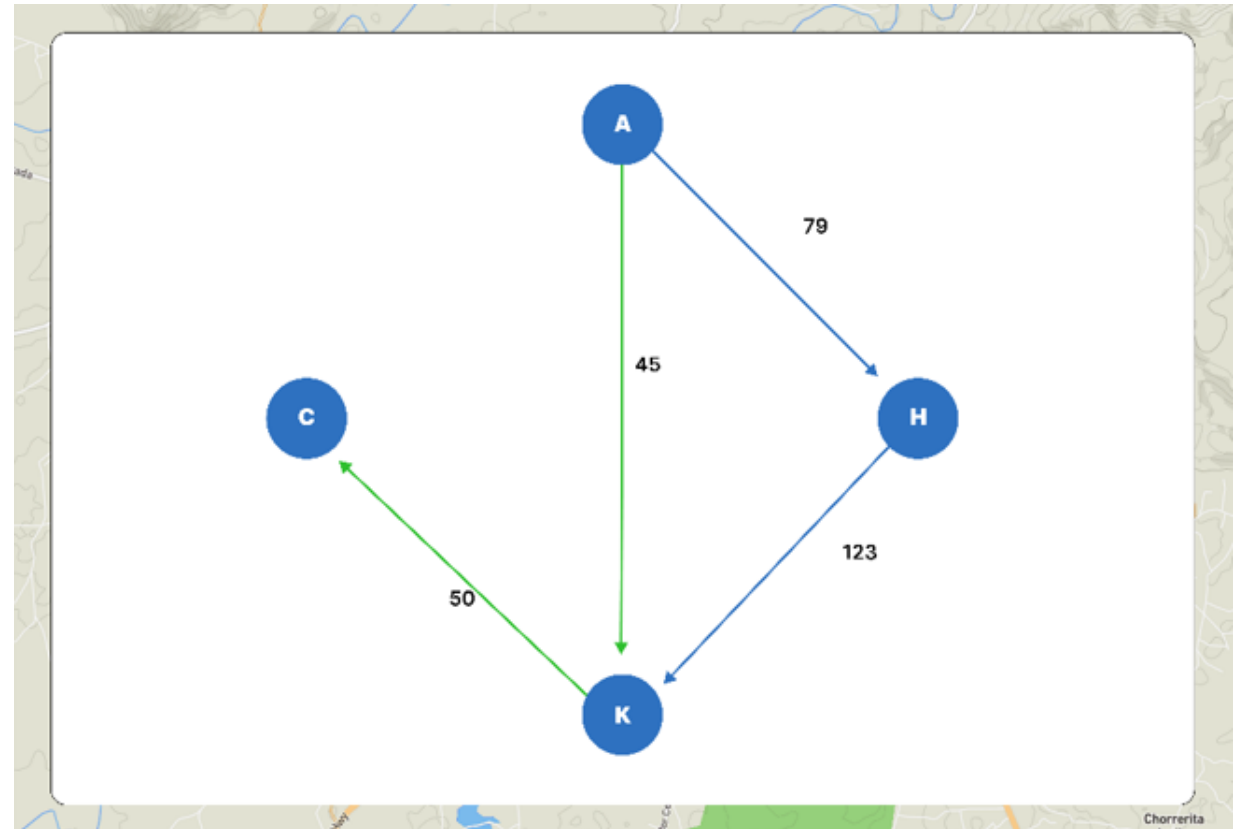
Marco teórico – estructura principal

Características

- Está formado por nodos y aristas.
- Puede ser dirigido o no dirigido.
- Permite analizar conexiones entre puntos.
- Facilita la búsqueda de rutas óptimas.
- Puede manejar grandes cantidades de información conectada.

Aplicación en el Proyecto

- Nodos → Intersecciones, semáforos y puntos de referencia.
- Aristas → Calles y vías de la ciudad.
- Pesos → Nivel de congestión vehicular.



Marco teórico – estructura principal

Gestión de Nodos

- Inserción de nodos.
- Edición de nodos.
- Eliminación de nodos.

Gestión de Aristas

- Creación de aristas.
- Búsqueda de conexiones.
- Eliminación de aristas.
- Modificación de pesos.

Matriz de Adyacencia

- Almacena las conexiones entre nodos.
- Guarda el peso asociado a cada ruta.
- Sirve de apoyo para los algoritmos de búsqueda y optimización.

Inserción de nodos

```
private void boton_agregar_nodoActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    nodo = espacio_ingresar_nodo.getText().trim();  
    if (!nodo.isEmpty()) {  
        // verificamos que el nombre no exista  
        for (int j = 0; j < grafo.obtenerNumeroDeNodos(); j++) {  
            if (grafo.obtenerNodos()[j].obtenerNombre().equals(nodo)) {  
                JOptionPane.showMessageDialog(this, "El nodo '" + nodo + "'  
                    + "ya existe", "Error", JOptionPane.WARNING_MESSAGE);  
                espacio_ingresar_nodo.setText("");  
                return;  
            }  
        }  
  
        grafo.agregarNodo(indice_nodo, nodo); // pasamos los nodos a grafo  
  
        combobox_nodo_salida.addItem(nodo); //añadimos la información al combo box  
        combobox_nodo_destino.addItem(nodo); //añadimos la información al combo box  
        indice_nodo++;  
  
        espacio_ingresar_nodo.setText(""); // limpia el espacio para el  
  
        grafo_interfaz.repaint();  
        actualizarInfoRed();  
    } else {  
        JOptionPane.showMessageDialog(this,  
            "Ingrese un nombre para el punto", "Error", JOptionPane.WARNING_MESSAGE);  
    }  
}
```

Matriz de adyacencia

Matriz de adyacencia guarda la información de las distancias entre los nodos siendo las filas los nodos de salida y las columnas los nodos de destino.

Se involucra en la búsqueda de ruta, ya que al almacenar la ruta se llama junto a otros métodos para mayor rapidez.

```
//obtener matriz de adyacencia para mostrar
public double[][] obtenerMatrizAdyacenciaParaMostrar() {
    double[][] mostrar = new double[numNodos][numNodos];
    for (int i = 0; i < numNodos; i++) { //recorrremos las filas de la matriz
        for (int j = 0; j < numNodos; j++) { //recorremos las columnas de la matriz
            mostrar[i][j] = matrizAdyacencia[i][j]; //copia la matriz de adyacencia original
        }
    }
    return mostrar;
}
```

```
//verificar si existe arista entre 2 vértices
public boolean existeArista(int origenId, int destinoId) {
    //obtenemos id
    int idxOrigen = obtenerIndiceNodo(origenId);
    int idxDestino = obtenerIndiceNodo(destinoId);

    if (idxOrigen == -1 || idxDestino == -1) { //validamos
        return false;
    }

    return matrizAdyacencia[idxOrigen][idxDestino] != Double.POSITIVE_INFINITY; //retornamos el id
}
```

```
//obtener el peso de las aristas
public double obtenerPesoArista(int origenId, int destinoId) {
    int idxOrigen = obtenerIndiceNodo(origenId);
    int idxDestino = obtenerIndiceNodo(destinoId);
    if (idxOrigen == -1 || idxDestino == -1) return Double.POSITIVE_INFINITY;
    return matrizAdyacencia[idxOrigen][idxDestino];
}
```


Marco teórico – estructura principal

Editar nodos

```
private void boton_eliminar_nodoActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    String nombre = espacio_ingresar_nodo.getText().trim();  
    if (nombre.isEmpty()) {  
        JOptionPane.showMessageDialog(this, "Ingrese el nombre del nodo a eliminar", "Error", JOptionPane.  
            return;  
    }  
    // Verificar existencia  
    Nodo[] nodos = grafo.obtenerNodos();  
    boolean existe = false;  
    for (int i = 0; i < grafo.obtenerNumeroDeNodos(); i++) {  
        if (nodos[i].obtenerNombre().equals(nombre)) { existe = true; break; }  
    }  
    if (!existe) {  
        JOptionPane.showMessageDialog(this, "El nodo '" + nombre + "' no existe", "Error", JOptionPane.WAI  
        espacio_ingresar_nodo.setText("");  
        return;  
    }  
    // Confirmar  
    int confirm = JOptionPane.showConfirmDialog(this,  
        "¿Eliminar el nodo '" + nombre + "'? Se eliminarán todas sus aristas.",  
        "Confirmar eliminación", JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);  
    if (confirm != JOptionPane.YES_OPTION) {  
        espacio_ingresar_nodo.setText("");  
        return;  
    }  
    // Eliminar  
    boolean eliminado = grafo.eliminarNodo(nombre);  
    if (eliminado) {  
        JOptionPane.showMessageDialog(this, "Nodo '" + nombre + "' eliminado correctamente", "Éxito", J  
        actualizarCombosLocales();  
        actualizarInfoRed();  
        grafo_interfaz.repaint();  
        espacio_ingresar_nodo.setText("");  
    } else {  
        JOptionPane.showMessageDialog(this, "Error al eliminar el nodo", "Error", JOptionPane.ERROR_MES  
    }  
}
```

Eliminar nodos

```
private void boton_eliminar_nodoActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    String nombre = espacio_ingresar_nodo.getText().trim();  
    if (nombre.isEmpty()) {  
        JOptionPane.showMessageDialog(this, "Ingrese el nombre del nodo a eliminar", "Error", JOptionPane.  
        return;  
    }  
    // Verificar existencia  
    Nodo[] nodos = grafo.obtenerNodos();  
    boolean existe = false;  
    for (int i = 0; i < grafo.obtenerNumeroDeNodos(); i++) {  
        if (nodos[i].obtenerNombre().equals(nombre)) { existe = true; break; }  
    }  
    if (!existe) {  
        JOptionPane.showMessageDialog(this, "El nodo '" + nombre + "' no existe", "Error", JOptionPane.WAI  
        espacio_ingresar_nodo.setText("");  
        return;  
    }  
    // Confirmar  
    int confirm = JOptionPane.showConfirmDialog(this,  
        "¿Eliminar el nodo '" + nombre + "'? Se eliminarán todas sus aristas.",  
        "Confirmar eliminación", JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);  
    if (confirm != JOptionPane.YES_OPTION) {  
        espacio_ingresar_nodo.setText("");  
        return;  
    }  
    // Eliminar  
    boolean eliminado = grafo.eliminarNodo(nombre);  
    if (eliminado) {  
        JOptionPane.showMessageDialog(this, "Nodo '" + nombre + "' eliminado correctamente", "Éxito", J  
        actualizarCombosLocales();  
        actualizarInfoRed();  
        grafo_interfaz.repaint();  
        espacio_ingresar_nodo.setText("");  
    } else {  
        JOptionPane.showMessageDialog(this, "Error al eliminar el nodo", "Error", JOptionPane.ERROR_MES  
    }  
}
```

Marco teórico – estructura principal

Algoritmo de Dijkstra

Es un algoritmo utilizado para encontrar el camino más corto entre un nodo de origen y uno o varios nodos de destino dentro de un grafo con pesos no negativos.

Funcionamiento

- Analiza las diferentes rutas posibles.
- Compara los costos acumulados de cada recorrido.
- Selecciona la alternativa de menor costo.
- Actualiza la información hasta encontrar la mejor ruta disponible.

Aplicación en el Proyecto

Dijkstra busca y toma la ruta más eficiente considerando el nivel de congestión vehicular de cada vía.

Resultado: se generan recorridos con menor tráfico y tiempos de desplazamiento más favorables.

Algoritmo de Dijkstra

```
public ResultadoDijkstra dijkstra(int origenId) {
    int idxOrigen = obtenerIndiceNodo(origenId);
    if (idxOrigen == -1 || numNodos == 0) return null;

    double[] dist = new double[numNodos];
    int[] padre = new int[numNodos];
    boolean[] visitado = new boolean[numNodos];

    Arrays.fill(dist, Double.POSITIVE_INFINITY);
    Arrays.fill(padre, -1);
    dist[idxOrigen] = 0;

    for (int i = 0; i < numNodos; i++) {
        // Seleccionar el nodo no visitado con menor distancia
        int u = -1;
        double minDist = Double.POSITIVE_INFINITY;
        for (int j = 0; j < numNodos; j++) {
            if (!visitado[j] && dist[j] < minDist) {
                minDist = dist[j];
                u = j;
            }
        }
        if (u == -1) break; // no hay más nodos alcanzables

        visitado[u] = true;

        // Relajar aristas desde u
        for (int v = 0; v < numNodos; v++) {
            if (!visitado[v] && matrizAdyacencia[u][v] != Double.POSITIVE_INFINITY) {
                double nuevaDist = dist[u] + matrizAdyacencia[u][v];
                if (nuevaDist < dist[v]) {
                    dist[v] = nuevaDist;
                    padre[v] = u;
                }
            }
        }
    }

    return new ResultadoDijkstra(dist, padre);
}
```

Marco teórico – estructura auxiliar

Hash

Una tabla hash es una estructura de datos utilizada para almacenar información mediante pares clave-valor. Permite realizar operaciones de inserción, búsqueda y eliminación de forma rápida, utilizando una función hash que transforma una clave en una posición específica dentro de una tabla.

Características

- Organiza la información mediante claves únicas asociadas a valores.
- Permite acceder rápidamente a los datos almacenados.
- Utiliza funciones hash para calcular posiciones de almacenamiento.
- Optimiza el rendimiento al manejar grandes cantidades de información.
- No mantiene un orden secuencial de los elementos.

Estructura de Hash

```
public class EstructuraHash<K, V> {  
    private static class NodoHash<K, V> {  
        K clave;  
        V valor;  
        NodoHash<K, V> siguiente;  
        NodoHash(K clave, V valor) {  
            this.clave = clave;  
            this.valor = valor;  
            this.siguiente = null;  
        }  
    }  
  
    private ArrayList<NodoHash<K, V>> tabla;  
    private int tamaño;  
    private int capacidad; // capacidad actual de la tabla  
    private ArrayList<V> duplicados; // lista de valores rechazados por clave duplicada  
    private boolean esSet; // true si se comporta como HashSet (solo claves)
```

Marco teórico – estructura auxiliar

Inserción

La operación de inserción se encarga de almacenar un nuevo nodo dentro de la tabla hash. Cuando se registra un elemento, se le asigna una clave que permite ubicarlo de forma rápida dentro de la estructura.

Búsqueda

La búsqueda permite recuperar la información de un nodo mediante su identificador. Esto evita recorrer todos los elementos almacenados y reduce significativamente el tiempo de acceso a los datos.

Funcionamiento

- Se genera una clave para cada nodo.
- La función hash calcula su posición.
- El nodo se almacena en la tabla.
- Cuando se realiza una consulta, el sistema utiliza la misma clave para localizar la información.

Método para agregar nodo

```
//agregamos un par clave-valor; si la clave ya existe, el valor se rechaza
public void agregar(K clave, V valor) {
    int indice = obtenerIndice(clave);
    NodoHash<K, V> cabeza = tabla.get(indice);
    NodoHash<K, V> actual = cabeza;

    // Verificar si la clave ya existe
    while (actual != null) {
        if (actual.clave.equals(clave)) {
            // Clave duplicada: guardar el valor en duplicados y salir
            duplicados.add(valor);
            return;
        }
        actual = actual.siguiente;
    }

    // clave nueva, insertar al inicio de la lista
    NodoHash<K, V> nuevo = new NodoHash<>(clave, valor);
    nuevo.siguiente = cabeza;
    tabla.set(indice, nuevo);
    tamaño++;
}
```


Marco teórico – estructura auxiliar

¿Qué es una colisión?

Una colisión ocurre cuando dos claves diferentes generan la misma posición dentro de la tabla hash. Para evitar conflictos, se utilizan técnicas de resolución de colisiones.

Método del Residuo

Este método calcula la posición inicial donde se almacenará un elemento mediante una operación módulo.

Fórmula:

Posición = Clave % TamañoTabla

Sondeo Cuadrático

Cuando una posición ya está ocupada, el sistema busca nuevas ubicaciones utilizando incrementos cuadráticos hasta encontrar una casilla libre.

Resultado

Permite mantener la eficiencia de la tabla hash incluso cuando existen conflictos de almacenamiento.

Función de residuo

```
//mostramos el cálculo del índice usando la clave (residuo) y también usando el tamaño para encontrar las colisiones con la
public String mostrarFuncionResiduo() {
    if (tamaño == 0) {
        return "No hay claves cargadas para mostrar la función residuo.";
    }
    StringBuilder mensaje = new StringBuilder();
    mensaje.append("=== FUNCIÓN RESIDUO ===\n");
    mensaje.append("Capacidad de la tabla: ").append(capacidad).append("\n");
    mensaje.append("Número de elementos insertados (tamaño): ").append(tamaño).append("\n\n");

    for (K clave : obtenerClaves()) {
        Integer claveInt = (Integer) clave;
        int indiceCapacidad = Math.abs(claveInt) % capacidad;
        int indiceTamaño = (tamaño > 0) ? Math.abs(claveInt) % tamaño : -1;

        mensaje.append("Clave: ").append(clave).append("\n");
        mensaje.append("  Cálculo residuo = ").append(claveInt).append(" % ").append(capacidad)
            .append(" = ").append(indiceCapacidad).append("\n");
        mensaje.append("  índice con capacidad (").append(capacidad).append(") = ").append(indiceCapacidad).append("\n");
        if (tamaño > 0) {
            mensaje.append("  Cálculo residuo = ").append(claveInt).append(" % ").append(tamaño)
                .append(" = ").append(indiceTamaño).append("\n");
            mensaje.append("  índice con tamaño (").append(tamaño).append(") = ").append(indiceTamaño).append("\n");
        } else {
            mensaje.append("  índice con tamaño: (tamaño = 0, no se puede calcular)\n");
        }
        mensaje.append("\n");
    }
    return mensaje.toString();
}
```


Grafo Dirigido (Principal)

- ▶ **Red Vial:** Representa la red de calles de la zona central de Penonomé.
- ▶ **Nodos (Vértices):** Puntos importantes del mapa, como intersecciones, avenidas y semáforos.
- ▶ **Aristas:** Son las calles o rutas disponibles que conectan los puntos.
- ▶ **Ponderación:** Nivel de congestión vehicular asignado (0 a 1000) para calcular rutas eficientes.

```
public class Grafo {  
    private int maxNodos;  
    private int numNodos;  
    private Nodo[] nodos;  
    private double[][] matrizAdyacencia;  
    private boolean esDirigido;  
    private EstructuraHash<Integer, Nodo> nodosHash;  
    private EstructuraHash<Integer, Integer> indicePorId;  
  
    // Constructor  
    public Grafo(int maxNodos, boolean esDirigido) {  
        this.maxNodos = maxNodos;  
        this.numNodos = 0;  
        this.esDirigido = esDirigido;  
        this.nodos = new Nodo[maxNodos];  
        this.matrizAdyacencia = new double[maxNodos][maxNodos];  
        this.nodosHash = new EstructuraHash<>();  
        this.indicePorId = new EstructuraHash<>();  
        // inicializamos la matriz con infinito (sin conexión)  
        for (int i = 0; i < maxNodos; i++) {  
            for (int j = 0; j < maxNodos; j++) {  
                matrizAdyacencia[i][j] = Double.POSITIVE_INFINITY;  
            }  
        }  
    }  
}
```

Tabla Hash (Auxiliar)

Estructura de Acceso

Almacena y recupera los nodos utilizando identificadores únicos que se generan automáticamente.

Permite acceder rápidamente a cualquier información específica sin tener que recorrer secuencialmente todo el grafo.

Atributos y Eficiencia

Mapea las claves asociadas y guarda las posiciones gráficas utilizadas por la interfaz del sistema.

Mejora sustancialmente el rendimiento de las búsquedas y agiliza los algoritmos de optimización de rutas.

Clases y Matriz de Adyacencia



Clase Nodo

Representa los vértices individuales de la ruta. Almacena el identificador genérico (ID), el nombre descriptivo y la posición en el espacio.



Matriz de Adyacencia

Estructura que guarda la información de las distancias. Almacena el peso específico del tráfico de cada arista conectada entre puntos.



Clase GrafoInterfaz

Componente visual fundamental. Se encarga de renderizar el grafo, interactuar con el usuario y dibujar rutas resaltadas en la pantalla.

Gestión de Nodos y Aristas

```
private void boton_buscar_aristaActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    String origen = (String) combobox_nodo_salida.getSelectedItem();  
    String destino = (String) combobox_nodo_destino.getSelectedItem();  
    if (origen == null || destino == null || origen.equals("(Sin nodos)") || destino.equals("(Sin nodos)")) {  
        JOptionPane.showMessageDialog(this, "Seleccione nodos válidos", "Error", JOptionPane.WARNING_MESSAGE);  
        return;  
    }  
    int idOrigen = obtenerIdPorNombre(origen);  
    int idDestino = obtenerIdPorNombre(destino);  
    if (idOrigen == -1 || idDestino == -1) {  
        JOptionPane.showMessageDialog(this, "Nodo no encontrado", "Error", JOptionPane.ERROR_MESSAGE);  
        return;  
    }  
    boolean existe = grafo.existeArista(idOrigen, idDestino);  
    double peso = grafo.obtenerPesoArista(idOrigen, idDestino);  
    if (existe) {  
        grafo_interfaz.resaltarArista(origen, destino);  
        textarea_impression.setText("Arista encontrada: " + origen + " → " + destino + " (peso: " + peso + ")");  
        // Mostrar en tabla  
        String[] columnas = {"Propiedad", "Valor"};  
        DefaultTableModel modelo = new DefaultTableModel(columnas, 0) {  
            @Override public boolean isCellEditable(int row, int col) { return false; }  
        };  
        modelo.addRow(new Object[]{"Origen", origen});  
        modelo.addRow(new Object[]{"Destino", destino});  
        modelo.addRow(new Object[]{"Peso", peso});  
        tabla_impression.setModel(modelo);  
    }  
}
```

- ▶ **Inserción y Edición:** Agrega elementos validando que no existan nombres duplicados y permite modificar atributos.
- ▶ **Creación de Aristas:** Conecta dos nodos generando un peso de congestión aleatorio simulando el tráfico real.
- ▶ **Búsqueda:** Verifica la existencia de aristas específicas y las resalta visualmente en el mapa.
- ▶ **Eliminación:** Remueve nodos o aristas de la red, actualizando automáticamente el estado y renderizado visual.

Dijkstra y Colisiones Hash



Algoritmo de Dijkstra

Calcula la ruta mínima entre puntos basándose en la congestión (peso).
Compara caminos y selecciona la alternativa de desplazamiento más fluida.



Método del Residuo

Operación matemática empleada por la función Hash para calcular y asignar la posición inicial exacta donde se almacenará cada nodo.

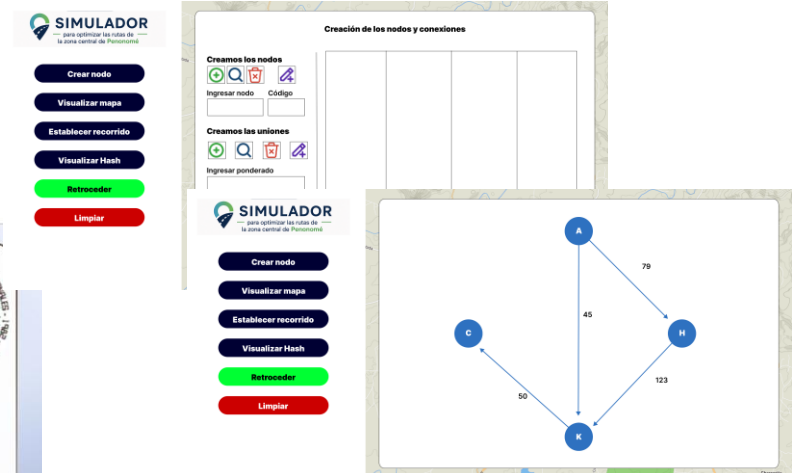


Sondeo Cuadrático

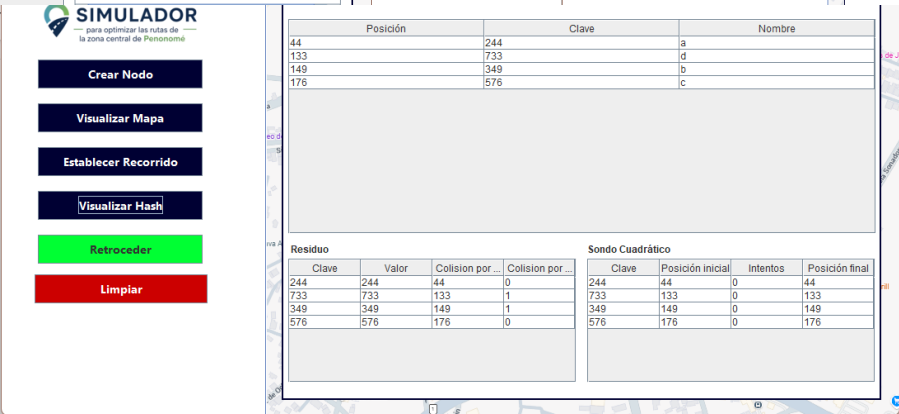
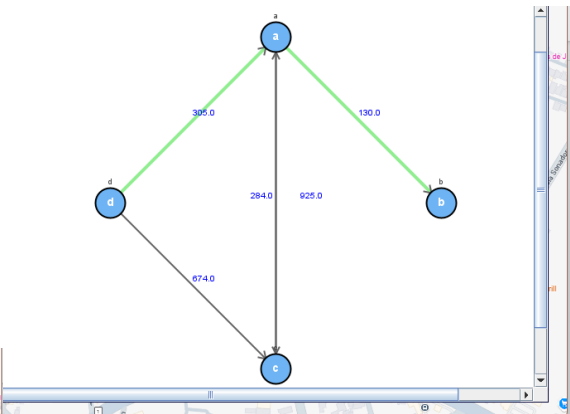
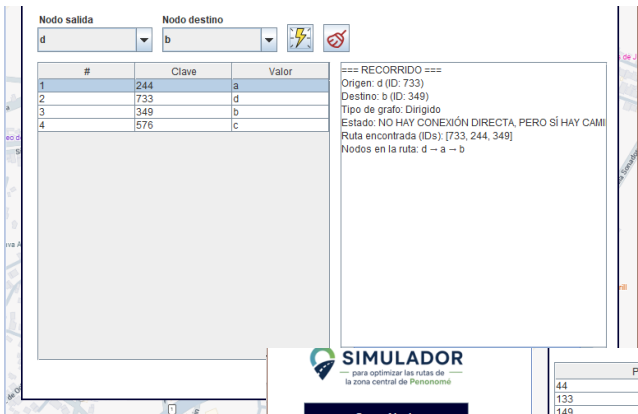
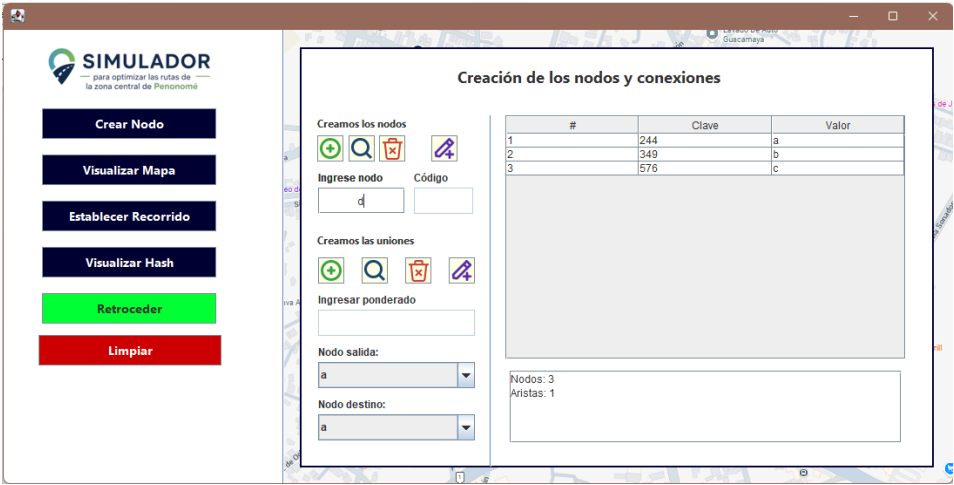
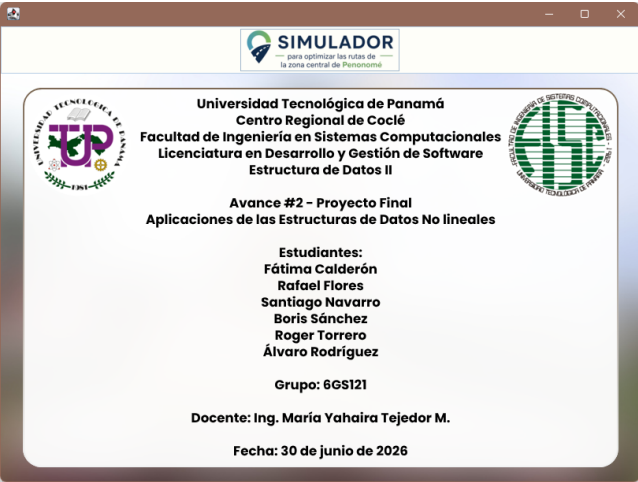
Mecanismo avanzado de resolución. Al existir colisiones, busca posiciones libres incrementando saltos cuadráticos para colocar el nodo.

Diseño Preliminar

El diseño preliminar del sistema fue desarrollado en Figma, creando un prototipo visual antes de su implementación. Incluye las principales pantallas del flujo de navegación, iniciando con una pantalla de bienvenida y presentación del proyecto. También incorpora interfaces para la creación y gestión de rutas, donde el usuario puede definir origen y destino, visualizar conexiones y observar la ruta óptima generada. El diseño busca una navegación sencilla e intuitiva que facilite la interacción y la comprensión de la red vial y las rutas sugeridas.



Ejecutando el programa



Conclusión

El desarrollo de este proyecto nos permitió aplicar los conceptos de estructuras de datos a una experiencia que puede ser observada en un entorno cotidiano, como el problema de congestionamiento vehicular por el cual atraviesan los vehículos en la zona central de Penonomé y en otros lugares. Gracias al simulador se consiguió modelar el comportamiento de la red de rutas y se pudo descubrir cómo la organización de la información se vincula de forma directa con la forma en la que se obtienen soluciones dentro de un sistema.

La utilización de grafos nos permitió realizar una representación de las conexiones de diferentes puntos del mapa y, además, trabajar con las relaciones de las rutas de una forma más estructurada. Por otro lado, la utilización de la tabla hash nos ayudó a facilitar el acceso a la información de los nodos a lo largo de la ejecución del sistema.

La utilización de algoritmos de recorridos y cálculo de las mejores rutas amparó permitió observar que la elección de una ruta no solo depende de un punto de partida y destino, sino que también de las relaciones que existentes y de los valores asociados al grado de congestión. El uso de algoritmo de búsqueda en profundidad y el algoritmo de Dijkstra ayudaron a trabajar la comparación entre diferentes opciones de recorrido.

Este proyecto permitió reforzar el uso de estructuras y algoritmos en una única solución de un problema aplicado en la vida real.

Referencias

- [1] M. Y. Tejedor, “2026_CAP 2_GRAFOS_2.1_2.2_2.3”, Estructura de Datos II, Facultad de Ingeniería en Sistemas Computacionales, Universidad Tecnológica de Panamá, Panamá, 19 de mayo de 2026. [Teams]. Disponible en: Plataforma Microsoft Teams. [Accedido: 20 de mayo de 2026].
- [2] M. Y. Tejedor, “2026_Final CAP GRAFOS_OperacionesRecorridos”, Estructura de Datos II, Facultad de Ingeniería en Sistemas Computacionales, Universidad Tecnológica de Panamá, Panamá, 1 de junio de 2026. [Teams]. Disponible en: Plataforma Microsoft Teams. [Accedido: 3 de junio de 2026].
- [3] UNADE, “Qué son los grafos”, UNADE Universidad, 23 de junio de 2025, [En línea]. Disponible: <https://unade.edu.mx/que-son-los-grafos/>. [Accedido: 12-may-2026].
- [4] P. Bello López, “Grafos,” Facultad de Ciencias de la Computación, Benemérita Universidad Autónoma de Puebla, Puebla, México, 2007. [En línea]. Disponible en: <https://www.cs.buap.mx/~pbello/grafos.pdf>. [Accedido: 12-may-2026].
- [5] Código Nautas, “Algoritmo Dijkstra: qué es y ejemplo,” [En línea]. Disponible en: <https://codigonautas.com/algoritmo-dijkstra-que-es-ejemplo/> [Accedido: 12-may-2026].
- [6] O. Cairó y S. Guardati, Estructura de Datos. México: McGraw-Hill, 2006.
- [7] L. Joyanes y I. Zahonero, Programación en algoritmos, estructuras de datos y objetos, 1.ª ed. Madrid, España: McGraw-Hill Interamericana de España, S.A., 2000.